SIMATS ENGINEERING

COMMON ASSESSMENT

UML-Based Design of an Autonomous Drone-Based Disaster Response and Rescue System (ADDRRS)

Course Name: Object Oriented Analysis and Design

Course Code: CSA1109

Assignment Weightage: 100%

Total Marks: 100

Name: Akash A

Reg: 192373015

GitHub: https://github.com/akkurthiakash/CSA1109

Code of Conduct

I, Akash A, declare that the work presented in this submission is entirely my own and that all guidelines set out for this assessment have been followed.

I acknowledge that breaching academic integrity rules will lead to disciplinary consequences.

Signature of the student: A Akash

1. Problem Analysis and Requirement Summary

1.1 Problem Statement

ADDRRS, the Autonomous Drone-Based Disaster Response and Rescue System, supports emergency responders during events such as floods, earthquakes, fires, and landslides. It relies on autonomous drones fitted with cameras and sensors to gather live information from affected zones, pinpoint the location of victims, track environmental conditions, and relay this data back to a central command centre.

Pairing autonomous aerial surveillance with a centralised command platform allows ADDRRS to cut down the gap between when a disaster is reported and when victims are found, so emergency coordinators can direct scarce rescue resources to where they will have the greatest impact.

1.2 Functional Requirements

- Register and manage disaster incidents.
- Define and manage disaster zones.
- Register and monitor drones.
- Assign drones to rescue missions.
- Plan and update drone routes.
- Collect images, videos, GPS locations, and sensor readings.
- Locate and record victims.
- Monitor drone status in real time.
- Generate emergency alerts.
- Coordinate emergency teams and resources.
- Handle drone failures and communication loss.

1.3 Non-Functional Requirements

- Real-time communication.
- High reliability and availability.
- Secure access for authorized personnel.
- Scalability for multiple disasters and drones.
- Fast response time.
- Maintainability and flexibility.
- Fault tolerance and failure handling.

2. Actor Identification

The following table sets out the primary and secondary actors involved with ADDRRS and outlines each one's responsibilities.

Actor	Responsibility
Emergency Coordinator	Registers incidents and manages rescue operations
Drone Operator	Monitors and controls drones when required
Rescue Team	Performs physical rescue operations
System Administrator	Manages users, drones, and system configuration
Autonomous Drone	Collects data and performs assigned missions
Central Command System	Processes and manages disaster information

Actor	Responsibility
Victim	Person identified as requiring rescue
Sensor System	Provides environmental readings
Notification Service	Sends emergency alerts and notifications

2. Use Case Descriptions

2.1 Major Use Cases

- Register Disaster Incident
- Define Disaster Zone
- Register Drone
- Assign Drone to Mission
- Plan Drone Route
- Launch Drone
- Monitor Drone
- Collect Sensor Data
- Capture Images/Videos
- Detect/Locate Victim
- Generate Alert
- Coordinate Rescue Team
- Manage Resources
- Handle Drone Failure

- Complete Rescue Mission

2.2 Example Use Case: Assign Drone to Rescue Mission

Field	Description
Use Case	Assign Drone to Rescue Mission
Primary Actor	Emergency Coordinator
Precondition	Disaster incident and drone are registered.
Main Flow	1. Coordinator selects a rescue mission. 2. System displays available drones. 3. Coordinator selects a suitable drone. 4. System assigns the drone and confirms the assignment.
Postcondition	The selected drone is assigned to the rescue mission and its status is updated to Assigned.

2.3 Use Case Diagram

The Use Case Diagram illustrates the boundary of ADDRRS, placing the principal use cases within it while the external actors sit outside the boundary, each linked to the relevant use cases through association lines.

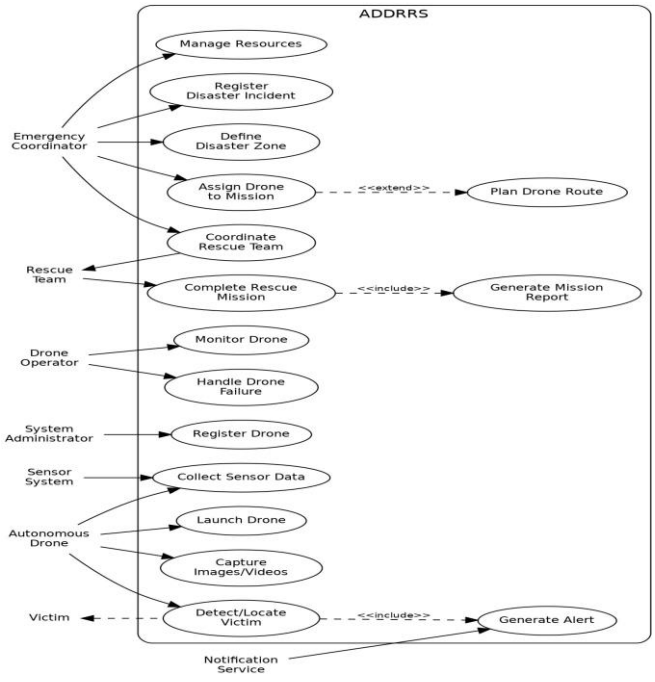


Figure 1: Use Case Diagram for ADDRRS

3. Object and Class Identification

3.1 Major Objects

Disaster, DisasterZone, Drone, DroneOperator, RescueMission, Victim, EmergencyTeam, SensorData, Alert, Resource, Route, MissionReport, EmergencyCoordinator.

3.2 Major Classes

Class	Important Attributes	Important Methods
Drone	droneId, status, location, batteryLevel	launch(), land(), move(), collectData()
Disaster	disasterId, type, severity, location	register(), updateStatus()
DisasterZone	zoneId, boundary, riskLevel	defineZone(), updateRisk()
RescueMission	missionId, status, priority	assignDrone(), startMission(), completeMission()
Victim	victimId, location, condition	updateCondition(), requestRescue()
EmergencyTeam	teamId, teamName, status	deploy(), performRescue()
SensorData	dataId, type, value, timestamp	collect(), transmit()
Alert	alertId, type, severity, message	generate(), send()
Route	routeId, startPoint, destination	calculate(), updateRoute()
Resource	resourceId, type, quantity	allocate(), release()
MissionReport	reportId, summary, timestamp	generate(), store()

4. Class Diagram

The Class Diagram brings together the classes described earlier and depicts how they relate to one another: Disaster connects to DisasterZone and RescueMission, RescueMission links to Drone, EmergencyTeam, Victim and MissionReport, and Drone connects to SensorData, Route and Alert. These relationships are represented through associations, aggregations, and dependencies, while a DroneManager and an EmergencyCoordinator control class are added to clarify how responsibilities are assigned.

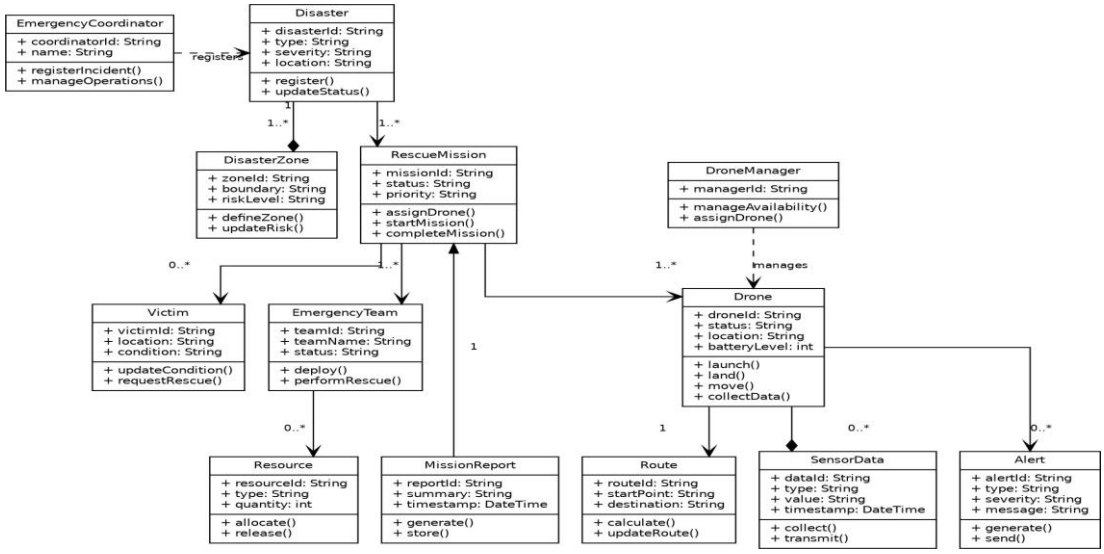


Figure 2: Class Diagram for ADDRRS

5. Sequence / Interaction Diagrams

Two sequence diagrams follow, each illustrating a typical flow of interactions within the system.

5.1 Sequence Diagram 1: Assign Drone to Rescue Mission

This diagram traces the steps where the Emergency Coordinator picks a mission, the Command System queries the Drone Manager for available drones, an appropriate drone is assigned, and confirmation of the assignment is sent back to the coordinator.

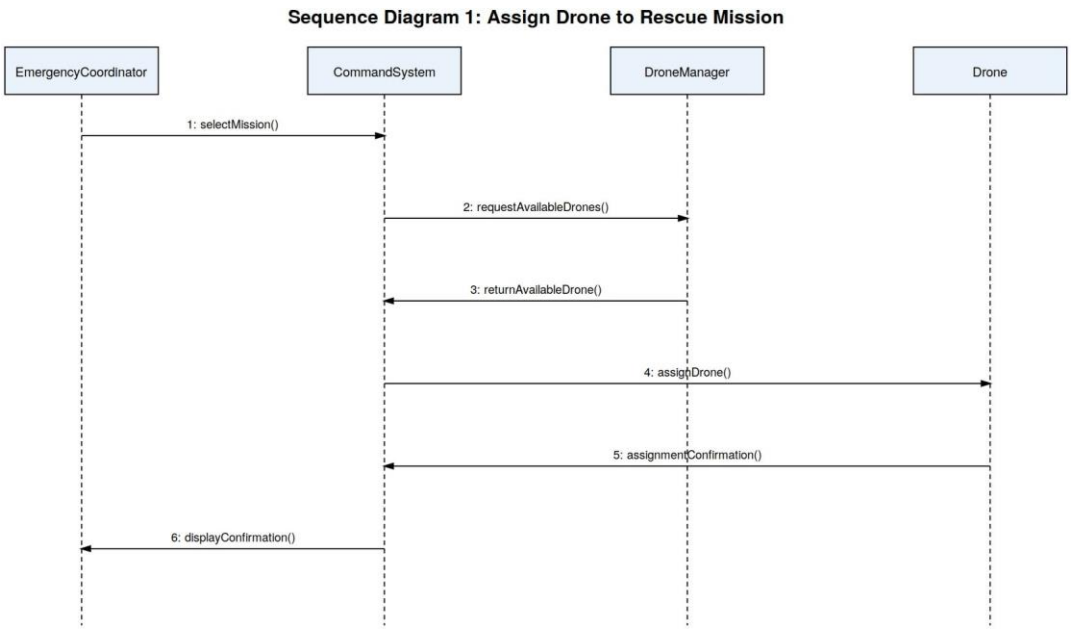


Figure 3: Sequence Diagram - Assign Drone to Rescue Mission

5.2 Sequence Diagram 2: Autonomous Victim Detection

This diagram traces how a drone's camera and sensors capture image data, which is passed to the Command System where the Victim Detection Module analyses it; once a victim is identified, the Alert Manager raises an alert and routes it to the Rescue Team.

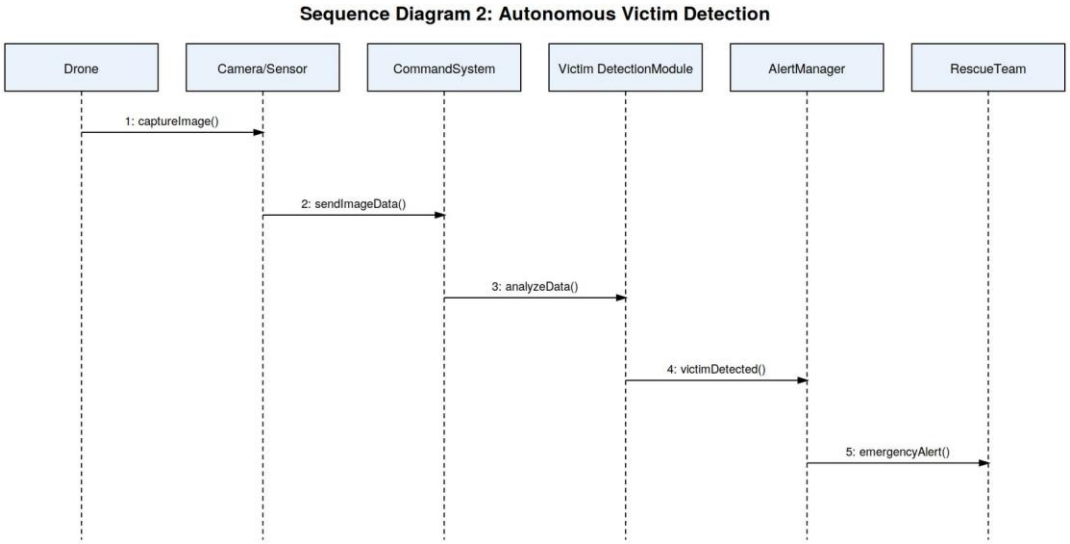
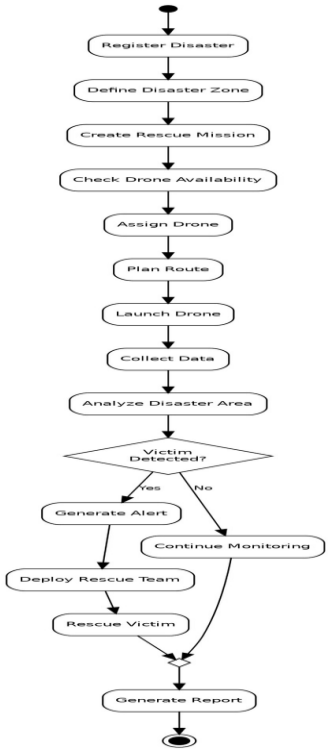


Figure 4: Sequence Diagram - Autonomous Victim Detection

6. Activity Diagram

The Activity Diagram maps out the complete Rescue Mission workflow, beginning with disaster registration and moving through drone assignment, data collection, victim detection, and finally the generation of a mission report. A decision node splits the flow depending on whether a victim is detected: if not, monitoring continues, and if so, the rescue process proceeds.

Figure 5: Activity Diagram - Rescue Mission Workflow



7. State Transition Diagram

Because a drone passes through a clearly defined lifecycle over the course of a mission, the Drone class is the best candidate for a State Transition Diagram. Its possible states include Registered, Available, Assigned, Ready, In Flight, Returning, Mission Completed, Low Battery, and Communication Loss.

Figure 6: State Transition Diagram for Drone

8. Package Diagram

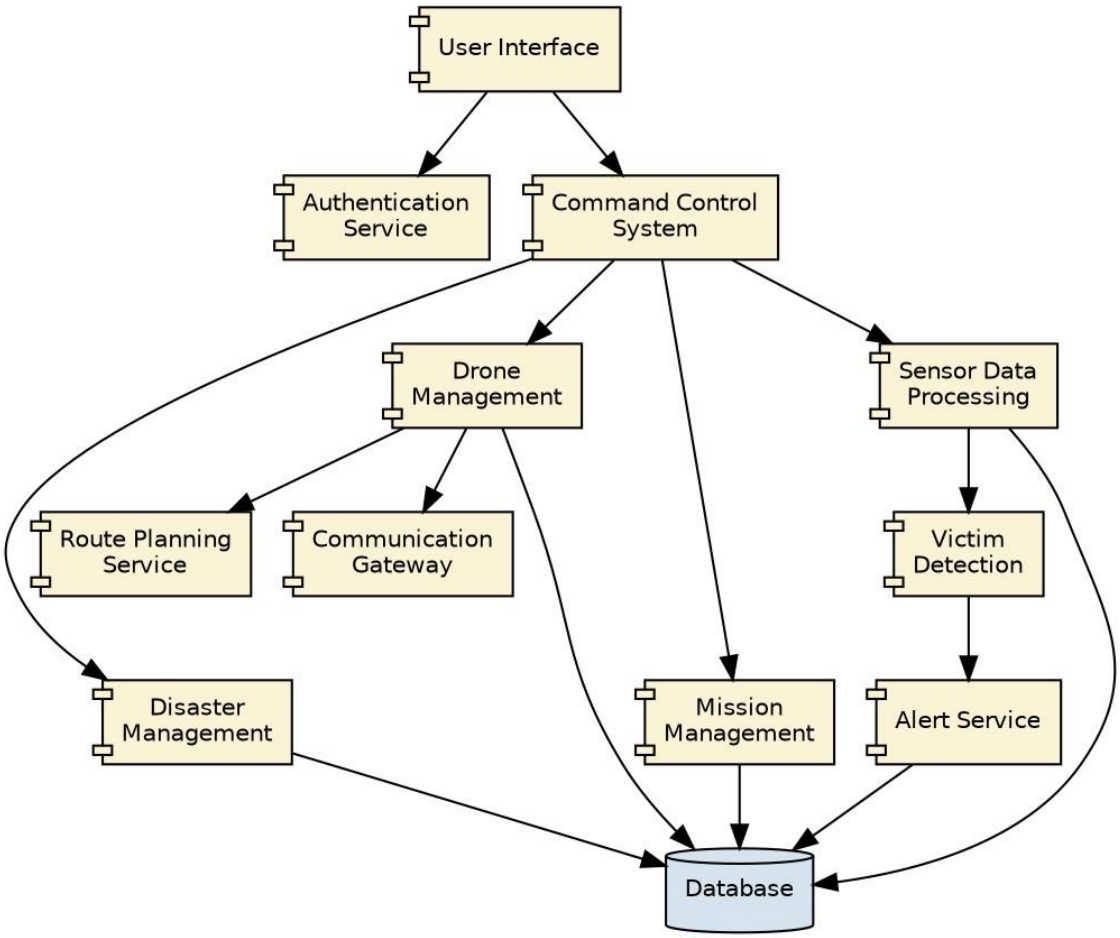
To keep the system modular and easy to maintain, ADDRRS is structured into the following packages: User Management, Disaster Management, Drone Management, Mission Management, Victim Management, Sensor & Data Management, Alert Management, Resource Management, Route Planning, Reporting, and Communication.

Figure 7: Package Diagram for ADDRRS

9. Component Diagram

ADDRRS is built from the following major components: User Interface, Authentication Service, Command Control System, Disaster Management, Drone Management, Mission Management, Route Planning Service, Sensor Data Processing, Victim Detection, Alert Service, Database, and Communication Gateway. As the diagram below shows, the Command Control System coordinates the various domain components, each of which stores its data in the shared Database.

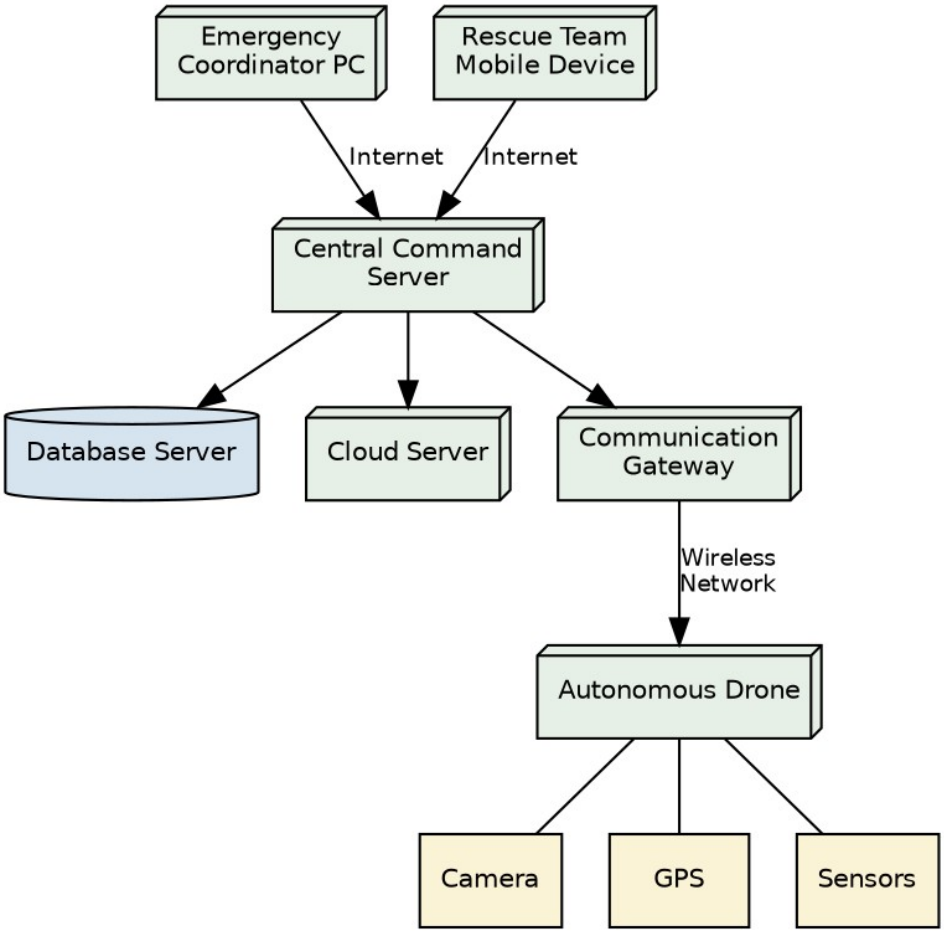
Figure 8: Component Diagram for ADDRRS



10. Deployment Diagram

The Deployment Diagram sets out the physical nodes that ADDRRS runs on: the Emergency Coordinator Computer and the Rescue Team Mobile Device both reach the Central Command Server over the Internet, and the Central Command Server in turn connects to the Database Server and the Cloud Server. The Central Command Server also communicates with the Autonomous Drones, each carrying a Camera, GPS, and Sensors, via a Communication Gateway over a wireless link.

Figure 9: Deployment Diagram for ADDRRS



11. Object Responsibility Analysis

Object/Class	Responsibility
Drone	Execute autonomous flight and collect information
RescueMission	Manage mission lifecycle
Disaster	Store disaster information
DisasterZone	Represent affected geographical area
Route	Calculate and manage drone routes
Victim	Maintain victim information
SensorData	Store environmental readings
Alert	Manage emergency notifications
EmergencyTeam	Perform rescue activities
Resource	Manage rescue resources
MissionReport	Generate mission completion information
DroneManager	Manage drone availability and assignments

11.1 Responsibility Assignment

High cohesion is achieved by keeping each class focused on a single responsibility — for instance, Route is limited to calculating and updating routes, and MissionReport only generates and stores reports — while low coupling is maintained by keeping dependencies between classes to a minimum; coordination logic sits within RescueMission and DroneManager instead of being spread across every domain class.

12. Design Axioms, Principles and Design Patterns

12.1 Design Principles

Principle	**Application in ADDRRS**
High Cohesion	Each class performs closely related operations (e.g. Drone only handles flight and data collection).
Low Coupling	Classes depend on interfaces/abstractions rather than unnecessary concrete implementations.
Single Responsibility Principle	Each class has one major responsibility (e.g. Alert only manages notifications).
Open/Closed Principle	New drone or sensor types can be added without major changes to existing code.
Encapsulation	Internal data such as drone status and battery level is protected through appropriate methods.

12.2 Suitable Design Patterns

Pattern	**Application**
Observer Pattern	Real-time drone status and alert notifications
Strategy Pattern	Different route-planning algorithms
Factory Pattern	Creating different drone/sensor objects
State Pattern	Managing drone states
Command Pattern	Sending commands to autonomous drones
Singleton Pattern	Central command/control manager, if a single shared instance is required

13. Assumptions and Constraints

13.1 Assumptions

- Only authorized personnel can access the system.
- Drones have GPS capability.
- Drones have cameras and environmental sensors.
- A communication network is available in most operational areas.
- Emergency teams have suitable communication devices.
- The central system maintains a database.

13.2 Constraints

- Limited drone battery capacity.
- Possible communication failures.
- Poor weather conditions may affect drone operation.
- GPS signals may become unavailable.
- Network bandwidth may be limited.
- System must handle multiple drones simultaneously.
- Security must prevent unauthorized access.
- Real-time data processing is required.

14. Requirement-to-Use Case/Class Traceability

Requirement	Use Case	Class(es)
Register disaster	Register Disaster Incident	Disaster
Define affected area	Define Disaster Zone	DisasterZone
Manage drones	Manage Drone	Drone, DroneManager
Assign drone	Assign Drone	Drone, RescueMission
Plan route	Plan Route	Route
Collect sensor data	Collect Data	Drone, SensorData
Locate victims	Locate Victim	Victim, Drone
Generate emergency notification	Generate Alert	Alert
Coordinate rescue	Coordinate Rescue	EmergencyTeam, RescueMission
Manage resources	Manage Resources	Resource
Complete mission	Complete Mission	RescueMission, MissionReport

15. Implementation / Prototype and Result

15.1 Prototype Scope

A basic prototype could showcase the following screens and functions:

- Login page
- Disaster registration
- Drone registration
- Drone status dashboard
- Mission assignment
- Disaster-zone display
- Victim information
- Sensor-data monitoring
- Emergency alerts

15.2 Expected Result

The prototype is expected to confirm that emergency staff are able to log disasters, assign drones, keep track of autonomous operations, receive the data that is gathered, identify victims, raise alerts, coordinate rescue teams, and bring rescue missions to completion.

16. Reflection on Design Decisions and Learning Outcomes

Building ADDRRS shows how Object-Oriented Analysis and Design principles can be applied to a genuine emergency-response scenario. Working through UML diagrams made it possible to pin down the system's actors, objects, classes, relationships, workflows, interactions, states, components, and deployment nodes.

The project provided practical understanding of:

- Object identification and classification.
- Use Case Modelling.
- UML diagram development.
- Class and relationship identification.
- Responsibility assignment.
- Object-oriented design principles.
- Design patterns.

GitHub Upload

https://github.com/akkurthiakash/CSA1109